

```
In [1]: def parse_sklearn_report(filepath):
        """
        Lê um arquivo de texto com a saída do sklearn.metrics.classification_
        e retorna as métricas em um dicionário.

        Args:
            filepath (str): O caminho para o arquivo de texto.

        Returns:
            dict: Um dicionário contendo as métricas por classe e as métricas
            Ex: {'0.0': {'precision': 0.9889, ...}, 'accuracy': 0.9728,
        """
        metrics = {}
        with open(filepath, 'r') as f:
            lines = f.readlines()

        class_metrics_started = False

        for line in lines:
            line = line.strip()

            # Ignora linhas vazias ou de cabeçalho
            if not line or 'precision' in line:
                continue

            parts = line.split()

            # Verifica se estamos nas métricas das classes
            if len(parts) == 5 and parts[0].replace('.', '', 1).isdigit():
                class_name = parts[0]
                metrics[class_name] = {
                    'precision': float(parts[1]),
                    'recall': float(parts[2]),
                    'f1-score': float(parts[3]),
                    'support': int(parts[4])
                }
                class_metrics_started = True

            # Verifica se estamos nas métricas agregadas
            elif class_metrics_started and len(parts) >= 2:
                key = ' '.join(parts[:-4]) if len(parts) > 4 else parts[0]
                key = key.replace(' ', '_')

                if 'accuracy' in key:
                    metrics['accuracy'] = float(parts[-2])
                elif 'macro' in key or 'weighted' in key:
                    metrics[key] = {
                        'precision': float(parts[-4]),
                        'recall': float(parts[-3]),
                        'f1-score': float(parts[-2]),
                        'support': int(parts[-1])
                    }

        return metrics
```

In [2]: `%pwd`

Out[2]: `'/home/antonio/Documents/programming/python/ML_code/SNN/09_Sep_2025'`

In [3]: `report_data = parse_sklearn_report('test_out_scikit_report.txt')`

In [4]: `print(report_data)`

```
{'0.0': {'precision': 0.9889, 'recall': 0.981, 'f1-score': 0.9849, 'support': 1000}, '1.0': {'precision': 0.8257, 'recall': 0.8911, 'f1-score': 0.8571, 'support': 101}, 'accuracy': 0.9728, 'macro_avg': {'precision': 0.9073, 'recall': 0.936, 'f1-score': 0.921, 'support': 1101}, 'weighted_avg': {'precision': 0.9739, 'recall': 0.9728, 'f1-score': 0.9732, 'support': 1101}}
```

Versão 2 com separação por médias

In [8]: `def parse_sklearn_report2(filepath):`

"""

Lê um arquivo de texto com a saída do `sklearn.metrics.classification_` e retorna as métricas em um dicionário. As chaves são renomeadas para serem mais explícitas (ex: `'precision'`, `'precision_macro_avg'`).

Args:

`filepath (str)`: O caminho para o arquivo de texto.

Returns:

`dict`: Um dicionário contendo as métricas por classe e as métricas
Ex: `{'0.0_precision': 0.9889, 'accuracy': 0.9728, ...}`

"""

`metrics = {}`

`with open(filepath, 'r') as f:`
 `lines = f.readlines()`

`class_metrics_started = False`

`for line in lines:`
 `line = line.strip()`

Ignora linhas vazias ou de cabeçalho

`if not line or 'precision' in line:`
 `continue`

`parts = line.split()`

Verifica se estamos nas métricas das classes

`if len(parts) == 5 and parts[0].replace('.', '', 1).isdigit():`
 `class_name = parts[0]`
 `metrics[f'{class_name}_precision'] = float(parts[1])`
 `metrics[f'{class_name}_recall'] = float(parts[2])`
 `metrics[f'{class_name}_f1-score'] = float(parts[3])`
 `metrics[f'{class_name}_support'] = int(parts[4])`
 `class_metrics_started = True`

Verifica se estamos nas métricas agregadas

`elif class_metrics_started and len(parts) >= 2:`
 `key_suffix = ''`

```
if 'accuracy' in line:
    metrics['accuracy'] = float(parts[-2])
elif 'macro avg' in line:
    key_suffix = 'macro_avg'
    metrics[f'precision_{key_suffix}'] = float(parts[-4])
    metrics[f'recall_{key_suffix}'] = float(parts[-3])
    metrics[f'f1-score_{key_suffix}'] = float(parts[-2])
    metrics[f'support_{key_suffix}'] = int(parts[-1])
elif 'weighted avg' in line:
    key_suffix = 'weighted_avg'
    metrics[f'precision_{key_suffix}'] = float(parts[-4])
    metrics[f'recall_{key_suffix}'] = float(parts[-3])
    metrics[f'f1-score_{key_suffix}'] = float(parts[-2])
    metrics[f'support_{key_suffix}'] = int(parts[-1])

return metrics
```

```
In [9]: report_data = parse_sklearn_report2('test_out_scikit_report.txt')
```

```
In [10]: report_data
```

```
Out[10]: {'0.0_precision': 0.9889,
          '0.0_recall': 0.981,
          '0.0_f1-score': 0.9849,
          '0.0_support': 1000,
          '1.0_precision': 0.8257,
          '1.0_recall': 0.8911,
          '1.0_f1-score': 0.8571,
          '1.0_support': 101,
          'accuracy': 0.9728,
          'precision_macro_avg': 0.9073,
          'recall_macro_avg': 0.936,
          'f1-score_macro_avg': 0.921,
          'support_macro_avg': 1101,
          'precision_weighted_avg': 0.9739,
          'recall_weighted_avg': 0.9728,
          'f1-score_weighted_avg': 0.9732,
          'support_weighted_avg': 1101}
```

```
In [ ]:
```